

Lab 7: Naive Bayes Classifier

Chirag Rao KV

240962180

AIML B 16

```
import numpy as np
import pandas as pd
import csv
from collections import defaultdict, Counter
import re, math
```

```
P_H = 0.60
P_D = 0.40
P_A_H = 0.30
P_A_D = 0.20
P_A = (P_A_H * P_H) + (P_A_D * P_D)
P_H_A = (P_A_H * P_H) / P_A
```

```
print("Probability that the student is a hosteler given A
grade:", P_H_A)
```

Probability that the student is a hosteler given A grade:
0.6923076923076923

```
P_D = 0.01
P_not_D = 0.99
P_pos_D = 0.99
P_pos_not_D = 0.02
P_pos = (P_pos_D * P_D) + (P_pos_not_D * P_not_D)
P_D_pos = (P_pos_D * P_D) / P_pos
```

```
print("Probability of having disease given positive test:", P_D_pos)
```

Probability of having disease given positive test: 0.3333333333333333

```
data = {
    "age": [
        "<=30", "<=30", "31...40", ">40", ">40", ">40",
        "31...40", "<=30", "<=30", ">40", "<=30",
        "31...40", "31...40", ">40"
    ],
    "income": [
        "high", "high", "high", "medium", "low", "low",
        "low", "medium", "low", "medium", "medium",
        "medium", "high", "medium"
    ]
}
```

```

],
"student": [
    "no", "no", "no", "no", "yes", "yes",
    "yes", "no", "yes", "yes", "yes",
    "no", "yes", "no"
],
"credit_rating": [
    "fair", "excellent", "fair", "fair", "fair", "excellent",
    "excellent", "fair", "fair", "fair", "excellent",
    "excellent", "fair", "excellent"
],
"buys_computer": [
    "no", "no", "yes", "yes", "yes", "no",
    "yes", "no", "yes", "yes", "yes",
    "yes", "yes", "no"
]
}

def naive_bayes(data, new_data):
    total = len(data["buys_computer"])
    class_count = defaultdict(int)
    for result in data["buys_computer"]:
        class_count[result] += 1
    probabilities = {}
    for target in ["yes", "no"]:
        probability = class_count[target] / total
        for feature in ["age", "income", "student", "credit_rating"]:
            count = 0
            for i in range(total):
                if (data[feature][i] == new_data[feature]
                    and data["buys_computer"][i] == target):
                    count += 1
            unique_values = len(set(data[feature]))
            probability *= (count + 1) / (class_count[target] +
unique_values)
        probabilities[target] = probability
    prediction = max(probabilities, key=probabilities.get)
    return prediction, probabilities

new_buyer = {"age": "<=30", "income": "medium", "student":
"yes", "credit_rating": "fair"}
prediction, probabilities = naive_bayes(data, new_buyer)
print("Probabilities:")
print("Yes =", probabilities["yes"])
print("No  =", probabilities["no"])
print("\nPrediction:", prediction)
if prediction == "yes":

```

```

    print("The buyer should BUY the computer.")
else:
    print("The buyer should NOT BUY the computer.")

Probabilities:
Yes = 0.027117768595041326
No  = 0.008199708454810493

Prediction: yes
The buyer should BUY the computer.

from collections import Counter

data = [
    ("A great game", "Sports"),
    ("The election was over", "Not sports"),
    ("Very clean match", "Sports"),
    ("A clean but forgettable game", "Sports"),
    ("It was a close election", "Not sports")
]

def naive_bayes(sentence):
    classes = ["Sports", "Not sports"]
    words = {c: [] for c in classes}

    for text, label in data:
        words[label] += text.lower().split()

    vocab = set(sum(words.values(), []))
    result = {}

    for c in classes:
        p = sum(x[1] == c for x in data) / len(data)

        for w in sentence.lower().split():
            p *= (words[c].count(w) + 1) / (len(words[c]) +
len(vocab))

        result[c] = p

    return max(result, key=result.get)

print("Prediction:", naive_bayes("A very close game"))
Prediction: Sports

```

Additional Questions

```

data = [
    ("Rainy", "Yes"), ("Sunny", "Yes"), ("Overcast", "Yes"),
    ("Overcast", "Yes"), ("Sunny", "No"), ("Rainy", "Yes"),

```

```

    ("Sunny", "Yes"), ("Overcast", "Yes"), ("Rainy", "No"),
    ("Sunny", "No"), ("Sunny", "Yes"), ("Rainy", "No"),
    ("Overcast", "Yes"), ("Overcast", "Yes")
]

train = data[:10]
test = data[10:]

def predict(weather, train):
    classes = ["Yes", "No"]
    result = {}
    for c in classes:
        rows = [x for x in train if x[1] == c]
        p = len(rows) / len(train)
        count = sum(x[0] == weather for x in rows)
        p *= (count + 1) / (len(rows) + 3)
        result[c] = p
    return max(result, key=result.get)

actual = [x[1] for x in test]
predicted = [predict(x[0], train) for x in test]
TP = sum(a == "Yes" and p == "Yes" for a, p in zip(actual, predicted))
TN = sum(a == "No" and p == "No" for a, p in zip(actual, predicted))
FP = sum(a == "No" and p == "Yes" for a, p in zip(actual, predicted))
FN = sum(a == "Yes" and p == "No" for a, p in zip(actual, predicted))
accuracy = (TP + TN) / len(test)
precision = TP / (TP + FP) if TP + FP else 0
recall = TP / (TP + FN) if TP + FN else 0
print("Predictions:", predicted)
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("Sunny ->", predict("Sunny", train))

Predictions: ['Yes', 'Yes', 'Yes', 'Yes']
Accuracy: 0.75
Precision: 0.75
Recall: 1.0
Sunny -> Yes

def train_naive_bayes(dataset):
    total = len(dataset)
    class_counts = Counter(row['golf'] for row in dataset)
    feature_counts = defaultdict(lambda: defaultdict(lambda: Counter()))

    for row in dataset:
        label = row['golf']
        for feat, val in row.items():
            if feat != 'golf':
                feature_counts[feat][val][label] += 1

```

```

return class_counts, feature_counts, total

def predict(sample, class_counts, feature_counts, total):
    probs = {}
    for label, count in class_counts.items():
        prob = count / total
        for feat, val in sample.items():
            num = feature_counts[feat][val][label] + 1
            den = count + len(feature_counts[feat])
            prob *= num / den
        probs[label] = prob
    return max(probs, key=probs.get)

data = [
    {'weather': 'rainy', 'temperature': 'hot', 'humidity': 'high',
     'windy': 'false', 'golf': 'no'},
    {'weather': 'rainy', 'temperature': 'hot', 'humidity': 'high',
     'windy': 'true', 'golf': 'no'},
    {'weather': 'overcast', 'temperature': 'hot', 'humidity': 'high',
     'windy': 'false', 'golf': 'yes'},
    {'weather': 'sunny', 'temperature': 'mild', 'humidity': 'high',
     'windy': 'false', 'golf': 'yes'},
    {'weather': 'sunny', 'temperature': 'cool', 'humidity': 'normal',
     'windy': 'false', 'golf': 'yes'},
    {'weather': 'sunny', 'temperature': 'cool', 'humidity': 'normal',
     'windy': 'true', 'golf': 'no'},
    {'weather': 'overcast', 'temperature': 'cool', 'humidity':
     'normal', 'windy': 'true', 'golf': 'yes'},
    {'weather': 'rainy', 'temperature': 'mild', 'humidity': 'high',
     'windy': 'false', 'golf': 'no'},
    {'weather': 'rainy', 'temperature': 'cool', 'humidity': 'normal',
     'windy': 'false', 'golf': 'yes'},
    {'weather': 'sunny', 'temperature': 'mild', 'humidity': 'normal',
     'windy': 'false', 'golf': 'yes'},
    {'weather': 'rainy', 'temperature': 'mild', 'humidity': 'normal',
     'windy': 'true', 'golf': 'yes'},
    {'weather': 'overcast', 'temperature': 'mild', 'humidity': 'high',
     'windy': 'true', 'golf': 'yes'},
    {'weather': 'overcast', 'temperature': 'hot', 'humidity':
     'normal', 'windy': 'false', 'golf': 'yes'},
    {'weather': 'sunny', 'temperature': 'mild', 'humidity': 'high',
     'windy': 'true', 'golf': 'no'}
]

class_counts, feature_counts, total = train_naive_bayes(data)
test_sample = {'weather': 'sunny', 'temperature': 'hot', 'humidity':
'high', 'windy': 'false'}

```

```
print('prediction:', predict(test_sample, class_counts,  
feature_counts, total))
```

```
prediction: no
```

```
data = {  
    "I love this sandwich": "pos",  
    "This is an amazing place": "pos",  
    "I feel very good about these beers": "pos",  
    "This is my best work": "pos",  
    "what a new awesome view": "pos",  
    "I do not like this restaurant": "neg",  
    "I am tired of this stuff": "neg",  
    "I can't deal with this": "neg",  
    "He is my sworn enemy": "neg",  
    "My boss is horrible": "neg",  
    "This is an awesome place": "pos",  
    "I don't like the taste of this juice": "neg",  
    "I love to dance": "pos",  
    "I am sick and tired of this place": "neg",  
    "what a great holiday": "pos",  
    "This is a bad locality to stay": "neg",  
    "we will have good fun tomorrow": "pos",  
    "I went to my enemy's house today": "neg"  
}
```

```
def words(s):  
    return re.findall(r'\w+', s.lower())
```

```
def train(data):  
    wc = {"pos":Counter(), "neg":Counter()}  
    total = {"pos":0, "neg":0}  
    docs = {"pos":0, "neg":0}  
  
    for text, c in data.items():  
        docs[c] += 1  
        for w in words(text):  
            wc[c][w] += 1  
            total[c] += 1  
  
    vocab = set(wc["pos"]) | set(wc["neg"])  
    return wc, total, docs, vocab
```

```
def predict(text, model):  
    wc, total, docs, vocab = model  
    scores = {}  
  
    for c in ["pos", "neg"]:  
        p = math.log(docs[c] / sum(docs.values()))  
        for w in words(text):
```

```

        if w in vocab:
            p += math.log((wc[c][w] + 1) /
                          (total[c] + len(vocab)))
        scores[c] = p

    return max(scores, key=scores.get)
items = list(data.items())
train_data = dict(items[:14])
test_data = dict(items[14:])

model = train(train_data)

tp = tn = fp = fn = 0

for text, actual in test_data.items():
    pred = predict(text, model)
    print(actual, pred)

    if actual == "pos" and pred == "pos": tp += 1
    elif actual == "neg" and pred == "neg": tn += 1
    elif actual == "neg" and pred == "pos": fp += 1
    else: fn += 1

print("Accuracy :", round((tp+tn)/4*100, 2), "%")
print("Precision:", round(tp/(tp+fp)*100, 2), "%")
print("Recall    :", round(tp/(tp+fn)*100, 2), "%")

pos pos
neg pos
pos pos
neg neg
Accuracy : 75.0 %
Precision: 66.67 %
Recall    : 100.0 %

```